



Reading the Docs, Writing the Errors



"Error 500. Something went wrong."

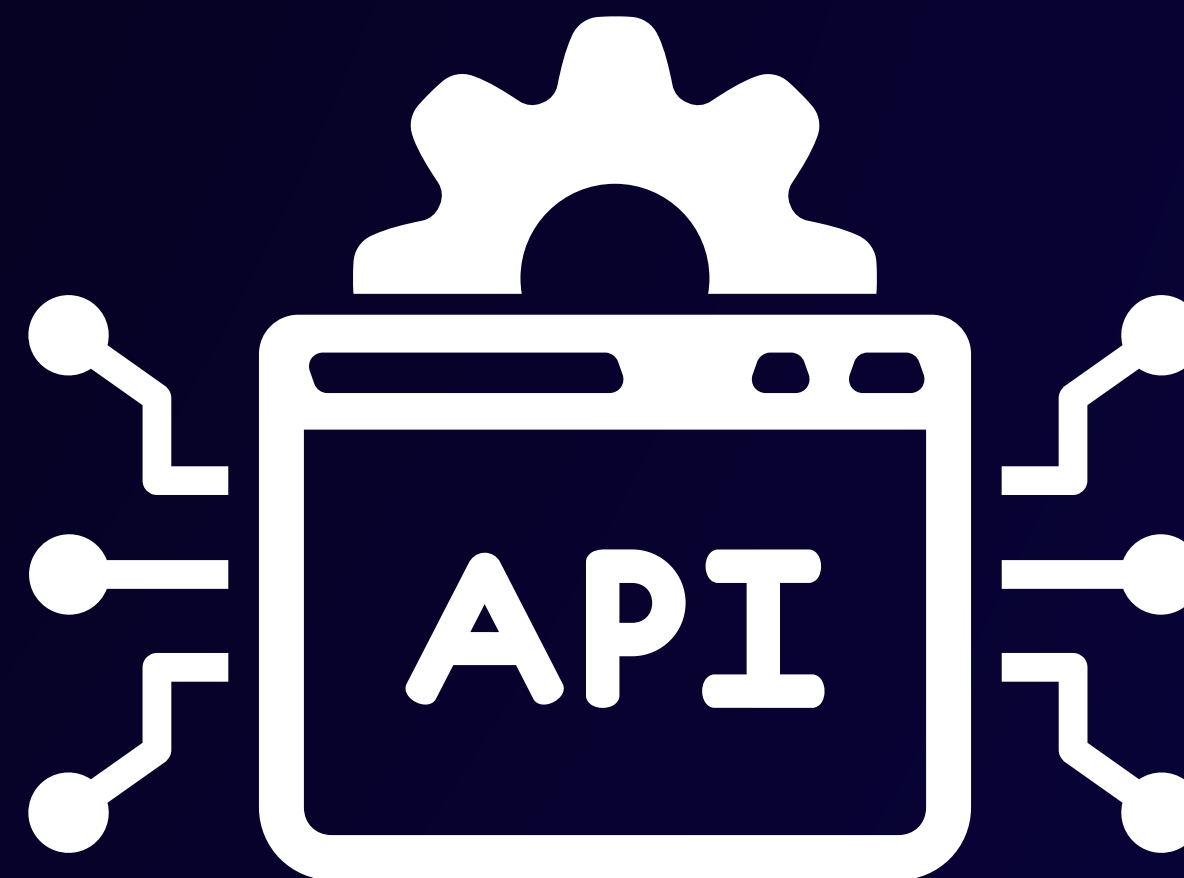
Co dokładnie poszło nie tak? **Nie wiadomo.**

Co ma teraz zrobić użytkownik? **Nie wiadomo.**



OpenAPI

OpenAPI Specification (OAS)—formerly known as Swagger—is a universally accepted, vendor-neutral standard for describing RESTful HTTP APIs. Written in JSON or YAML, it creates a machine-readable contract of your API's endpoints, parameters, and authentication methods.





OpenAPI – Struktura dokumentacji

paths

the URL patterns exposed by the API, e.g. `/tasks/{id}`

operation

a single combination of an HTTP method and a path,
e.g. `PUT /tasks/{id}`

parameters

inputs passed via the URL or headers (path, query, header)

requestBody

the data sent in the body of the request

responses

the possible outcomes of an operation, grouped by status
code



Termin	Znaczenie	Tłumaczenie
path parameter	a variable part of the URL path, e.g. {id} in /tasks/{id}	parametr ścieżki
query parameter	a value appended to a URL after "?" to filter or modify a request	parametr zapytania
request body	the data sent by the client in the body of the request	treść żądania
schema	a formal description of the structure and types of a piece of data	schemat danych
required / optional	whether a field must be present, or may be omitted	wymagane / opcjonalne
enum	a fixed set of allowed values for a field	zbiór dozwolonych wartości
operation	a single combination of an HTTP method and a path, e.g. POST /tasks	operacja (metoda + ścieżka)
authentication token	a piece of data proving the identity of the caller	token uwierzytelniający





Gdzie podróżują parametry?

path parameter

`/tasks/{id}`

part of the URL itself — identifies a specific resource

query parameter

`?status=open`

appended after "?" — filters or modifies the request

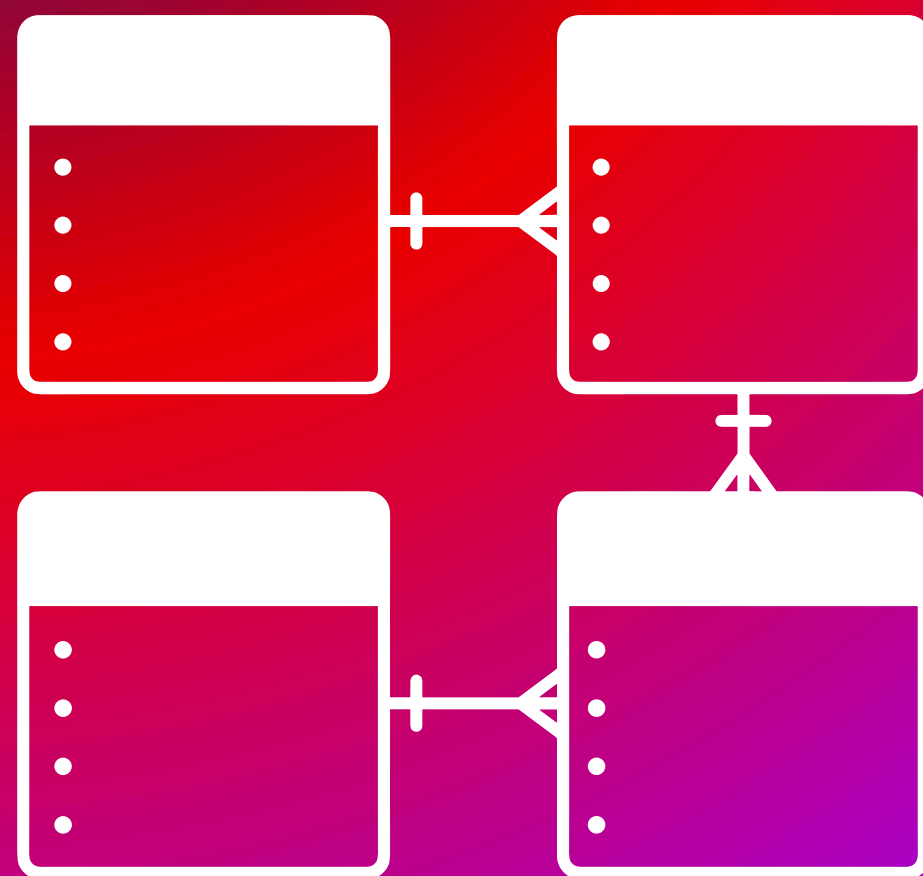
header

Authorization: Bearer ...

metadata sent alongside the request, not in the URL



schema



a formal description of the structure and types of a piece of data



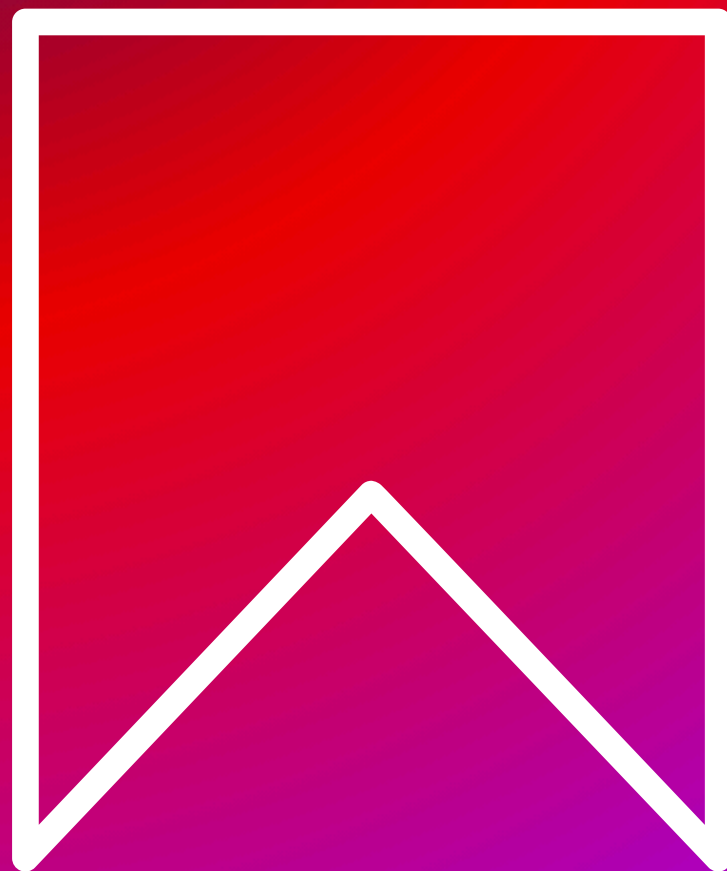
required /
optional



whether a field must be present, or
may be omitted



enum



a fixed set of allowed values for a field,
e.g. "open" | "in_progress" | "done"



Exercise – endpoint



POST /api/tasks/{id}/comments

Summary: Add a comment to an existing task.

Parameters:

id (path, string, required) – the task's unique identifier
Authorization (header, string, required) – Bearer token

Request body (application/json):

text (string, required) – the comment content
visibility (string, optional) – one of: "team", "private"
(default: "team")

Responses:

201 – comment created; returns the comment object
400 – the text field is missing or empty
404 – no task exists with the given id

1. Which HTTP method does this operation use?
2. Where is the id parameter located — path, query, or header?
3. Which THREE values is the status field allowed to have?
4. Which status code is returned if the Authorization header is missing?
5. Which status code is returned if the task does not exist?



RFC 7807

RFC 7807 is a technical standard created by the Internet Engineering Task Force (IETF) that defines a standardized, machine-readable format for returning error details from HTTP APIs.



RFC EDITOR
Official home of RFCs

RFC 7807 Problem Details for HTTP APIs

M. Nottingham, E. Wilde

This document defines a "problem detail" as a way to carry machine-readable details of errors in a HTTP response to avoid the need to define new error response formats for HTTP APIs.



RFC 7807 – pola obiektu błędu

Pole	Znaczenie
type	a URI that identifies the problem type (ideally links to documentation about it)
title	a short, human-readable summary of the problem type — stays the same for the same type
status	the HTTP status code for this occurrence of the problem
detail	a human-readable explanation specific to this occurrence of the problem
instance	a URI identifying this specific occurrence of the problem



RFC 7807 – pola obiektu błędu



HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```
{  
  "type": "https://api.taskflow.io/errors/invalid-status",  
  "title": "Invalid Status Value",  
  "status": 400,  
  "detail": "The status field must be one of: open, in_progress, done. Received: archived.",  
  "instance": "/api/tasks/482/status"  
}
```



How to write good error messages?

Be specific

name the exact field or condition that caused the problem, not just "error" or "invalid".

Be constructive

tell the user what to do next, not only what went wrong.

Avoid blaming the user

describe the condition ("the token is missing") rather than accusing ("you forgot the token").

Avoid vague, generic words on their own "error", "failed", "invalid", "unexpected error".

Use "please" only when asking the user to do something inconvenient
– not for standard actions



Exercise – error messages

Zły komunikat	Problem	Poprawiona wersja
"Error 500. Something went wrong."	<i>Vague — doesn't say what went wrong or what the user can do next.</i>	"We couldn't save your task because our server ran into a problem. Please try again in a few minutes."
"Invalid input."	<i>Doesn't specify which field is invalid or why.</i>	"The due date must be a date in the future."
"ERROR!!! Bad request."	<i>Unprofessional tone (all caps, exclamation marks) and no useful information.</i>	"We couldn't process your request because the 'status' field is missing."
"You entered the wrong value."	<i>Blames the user directly instead of stating the condition neutrally.</i>	"The status value 'archived' isn't supported. Use one of: open, in_progress, done."



APLIKACJE
WEBOWE

:wq

L1



by **Mateusz Węglewski**